



TFE4141 – Design of Digital Systems 1

Introductory Presentation of the Term Project

September 1, 2025

Outline

- Term project - goals
- RSA encryption algorithm
- Requirements and specification
- RSA Integration kit
- Term project report / datasheet
- Schedule

Term Project

Learning outcome

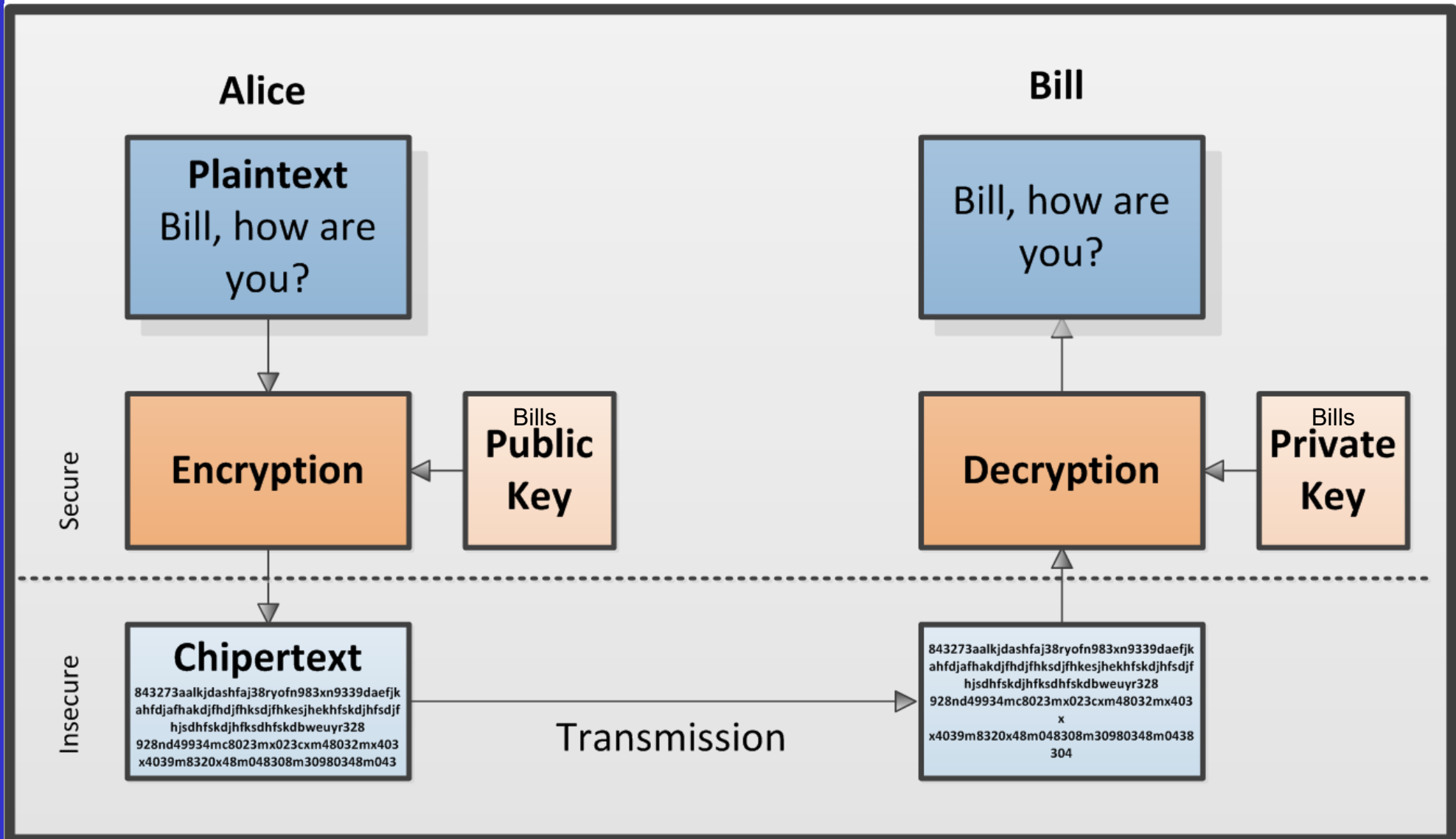
- **Important part of the training in TFE4141**
- **Design and get practical skills in using EDA tools**
- The work on this term project will give you experience with :
 - Implementation of algorithms in hardware
 - Verification by simulation
 - Use of tools for synthesis
 - To understand and evaluate other people's work by a peer-review system. This will also help you to receive critique (and praise..)
- **25 %** of the final grade is based upon the project report

RSA

Ron Rivest, Adi Shamir, Len Adleman

- Block cipher: both plaintext and cipher text are integers between 0 and $n-1$.
- RSA is an **asymmetric** algorithm using two different keys:
 - Public key for encryption
 - Private key for decryption
- Relies on a one-way function **f** :
 - easy to compute **$Y = f(X)$**
 - impossible to compute **$X = f^{-1}(Y)$** , if you don't have the correct key **k**
- Widely used in electronic commerce protocols, and is believed to be secure, given sufficiently long keys and the use of up-to-date implementations

RSA Encryption/Decryption Mechanism



RSA

- The **same computation** is performed for encryption and decryption, but with different parameters.
- The encryption and decryption keys (**d**, **e**) are related through the following equation:
- **Encryption**: $C = M^e \bmod n$, $M < n$
public key: {e, n}
- **Decryption**: $M = C^d \bmod n = (M^e)^d \bmod n$
private key: {d, n}
- $M = (M^e)^d \bmod n$ restricts the values for **e**, **d** and **n**.

Generating the keys

- 1) Select two prime numbers **p** and **q**
- 2) Compute **$n = pq$**
- 3) Compute **$\Phi(n) = \Phi(p) * \Phi(q) = (p - 1) (q - 1)$**
- 4) Select **e** such that **$\gcd(\Phi(n), e) = 1, 1 < e < \Phi(n)$**
- 5) Compute **$d = e^{-1} \bmod \Phi(n)$**

$\Phi(k)$: Euler's totient function

gcd: greatest common divisor

A simple example

Key generation:

- Find two primes $p = 7$, $q = 17$
- $n = pq = 7 \cdot 17 = 119$
- $\Phi(n) = 6 \cdot 16 = 96$
- Select e such that e and $\Phi(n)$ are relative primes. $e = 5$ can be used.
- Determine d such that $d = e^{-1} \pmod{96}$ and $d < 96$.
 $d = 77$ fulfills these requirements.
- Public key : $\{e, n\} = \{5, 119\}$
- Private key: $\{d, n\} = \{77, 119\}$

Encrypt/Decrypt:

- Message: $M = 19$
- Encryption: $19^5 \pmod{119} = 2476099 \pmod{119} = (20807 \cdot 119 + 66) \pmod{119} = 66$
- Decryption: $66^{77} \pmod{119} = \dots$ "Large numbers" $\dots = 19$

We need algorithm for $\underline{M}^e \bmod n$

1) Very naive method for computing M^e (brute force):

Say we were to compute M^{55} .

One possible solution:

```
C = M*M*M*M*M*M*M*M*M*M*
    M*M*M*M*M*M*M*M*M*M*
    M*M*M*M*M*M*M*M*M*M*
    M*M*M*M*M*M*M*M*M*M*
    M*M*M*M*M*M*M*M*M*M*
    M*M*M*M*M
```

This is an algorithm with complexity $O(e)$

We need algorithm for $M^e \bmod n$

2) A much better solution for computing M^e (repeated squaring)

The exponent written with different base numbers:

$e = "55"\text{dec} = "00110111"\text{bin}$

The exponent written in order to show the bit indexes

$e : 0 \quad 0 \quad 1 \quad 1 \quad 0 \quad 1 \quad 1 \quad 1$

$i : 7 \quad 6 \quad 5 \quad 4 \quad 3 \quad 2 \quad 1 \quad 0$

The exponent written as a sum of powers of two:

$$\begin{aligned} e &= 0 \cdot 2^7 + 0 \cdot 2^6 + 1 \cdot 2^5 + 1 \cdot 2^4 + 0 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 \\ &= 1 \cdot 2^5 + 1 \cdot 2^4 + 1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = 55 \end{aligned}$$

M^{55} can be rewritten like this:

$$M^{55} = M^{[1 \cdot 2^5 + 1 \cdot 2^4 + 1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0]}$$

$$= M^{[2^5 + 2^4 + 2^2 + 2^1 + 2^0]}$$

$$= M^{[32 + 16 + 4 + 2 + 1]}$$

$$= M^{32} * M^{16} * M^4 * M^2 * M^1$$

We need algorithm for $M^e \bmod n$

$M^{32} * M^{16} * M^4 * M^2 * M^1$ can be found as follows:

$$\begin{aligned} e_0 = 1: \quad C &= M = M^1 \\ P &= M * M = M^2 \end{aligned}$$

$$\begin{aligned} e_1 = 1: \quad C &= P * C = M^2 * M^1 \\ P &= P * P = M^4 \end{aligned}$$

$$\begin{aligned} e_2 = 1: \quad C &= P * C = M^4 * M^2 * M^1 \\ P &= P * P = M^8 \end{aligned}$$

$$\begin{aligned} e_3 = 0: \quad C &= C = M^4 * M^2 * M^1 \\ P &= P * P = M^{16} \end{aligned}$$

$$\begin{aligned} e_4 = 1: \quad C &= P * C = M^{16} * M^4 * M^2 * M^1 \\ P &= P * P = M^{32} \end{aligned}$$

$$\begin{aligned} e_5 = 1: \quad C &= P * C = M^{32} * M^{16} * M^4 * M^2 * M^1 \\ P &= P * P = M^{64} \end{aligned}$$

$$\begin{aligned} e_6 = 0: \quad C &= C = M^{32} * M^{16} * M^4 * M^2 * M^1 \\ P &= P * P = M^{128} \end{aligned}$$

$$\begin{aligned} e_7 = 0: \quad C &= C = M^{32} * M^{16} * M^4 * M^2 * M^1 \quad \leftarrow \text{CORRECT ANSWER} \\ P &= P * P = M^{256} \end{aligned}$$

Maximum 15 multiplications were needed here....

We need algorithm for $\underline{M}^e \bmod n$

Pseudocode for the algorithm:

```
C := 1
P := M
for i=0 to k-1
  if  $e_i = 1$ 
    C := C * P
  P := P * P
return C
```

This is an algorithm with complexity $O(\log e)$ which is a lot better than $O(e)$.

We now need algorithm for $A*B$

1) Paper and pencil method for multiplying two nonnegative integers:

A = 1001 = 9
 B = 1011 = 11
 R = A*B = 9*11 = 99

```

      1001 * 1011
      -----
        11
       1001
      1001
     0000
    1001
   -----
  = 1100011
   -----
   -----
    
```

$R = 1100011 = 2^6 + 2^5 + 2^1 + 2^0 = 64 + 32 + 2 + 1 = 99$

An algorithm for multiplying two nonnegative integers:

```

R := 0           | Initial value for partial sum
for i=0 to k-1   | k is the number of bits in B
    R = 2R + A*Bk-1-i | Bi is the i'th bit in B
return R
    
```

We now need algorithm for $A*B$

1) Example calculation using the algorithm on the previous slide:

$$A = 1001 = 9$$

$$B = 1011 = 11$$

$$R = A*B = 9*11 = 99 = 1100011$$

$$k = 4 \text{ (since } B \text{ is four bit wide)}$$

$$i=0 \Rightarrow k-1-i=3$$

$$R = 0 + 1001*1 = 1001$$

$$i=1 \Rightarrow k-1-i=2$$

$$R = 10010 + 1001*0 = 10010 \quad \text{--mult } R \text{ with } 2 \Rightarrow \text{shift left}$$

$$i=2 \Rightarrow k-1-i=1$$

$$R = 100100 + 1001*1 = 101101$$

$$i=3 \Rightarrow k-1-i=0$$

$$R = 1011010 + 1001*0 = 1100011$$

We need algorithm for $M^e \bmod n$

```
#    Pseduocode for the algorithm:
#
#    C := 1
#    P := M
#    for i=0 to k-1
#        if ei = 1
#            C := C*P
#            P := P*P
#    return C
#
#    This is an algorithm with
#    complexity  $O(\log e)$  which is a lot
#    better than  $O(e)$ .
```

Modular exponentiation:
 replace all multiplications by
 modular multiplications



Replace with modular multiplications

Let us look at $6 * 8 * 3$ used with (mod 5)

$$(6 * 8 * 3) = 144$$

$$144 = ? \pmod{5}$$

$$144/5 = 28.8$$

$$28 * 5 = 140$$

$$144 - 140 = 4$$

$$144 = \underline{4} \pmod{5}$$

$$\underline{6} = 1 \pmod{5}$$

$$(1 * 8) = 8 = 3 \pmod{5}$$

$$(3 * 3) = 9 = \underline{4} \pmod{5}$$

An example of how modular exponentiation is achieved by replacing all multiplications with modular multiplications

Replace $A*B$ with $A*B \bmod n$

```
# 2) Paper and pencil method for modular multiplication of two nonnegative
# integers:
```

```
# This can be done very similar to the algorithm explained in 1)
```

```
# R := 0                                | Initial value for partial sum
# for i=0 to k-1                        | k is the number of bits in B
#   R = 2R + A*Bk-1-i                  | Bi is the i'th bit in B, A <= N-1
#   R = R mod N                        | Interleaved reduction of R
# return R
```

```
# How to perform R = R mod N?
```

```
# The largest possible number produced by R mod N is: N-1
```

```
# The largest possible number produced by A*Bi is: N-1
```

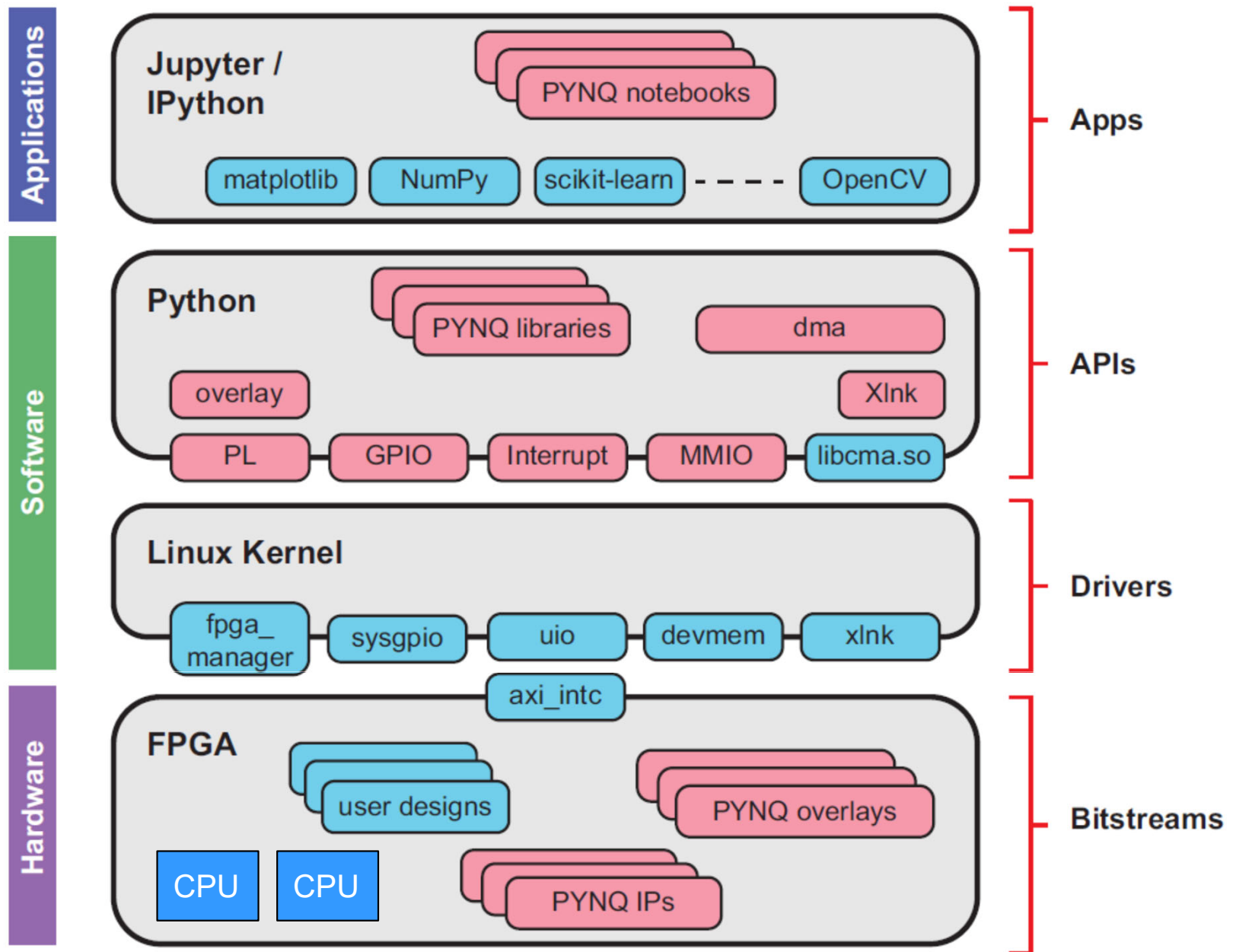
```
# Due to the interleaved reduction we will always have R in the range
```

```
# 0 <= R <= 2*(N-1) + (N-1) = 3N - 3
```

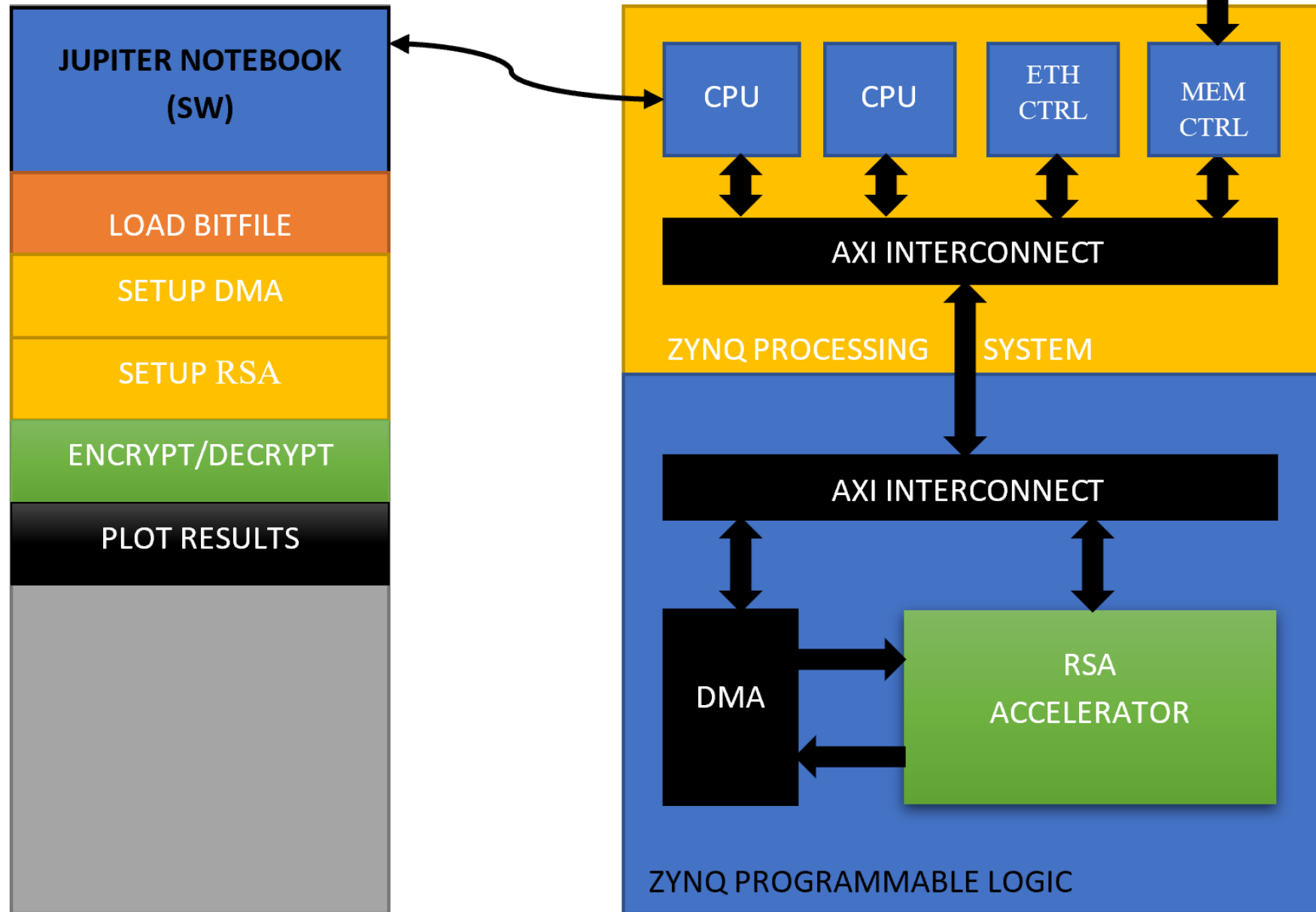
```
# In order to reduce R to the range 0 <= R <= N-1 it is necessary with
# at most two subtractions. The new pseudocode will then be:
```

```
# R := 0                                | Initial value for partial sum
# for i=0 to k-1                        | k is the number of bits in B
#   R = 2R + A*Bk-1-i                  | Bi is the i'th bit in B
#   if R >= N                           |
#     R = R - N                          |
#   if R >= N                           | Reduction ...
#     R = R - N                          |
# return R
```

General PYNQ Card and Framework



Our Framework



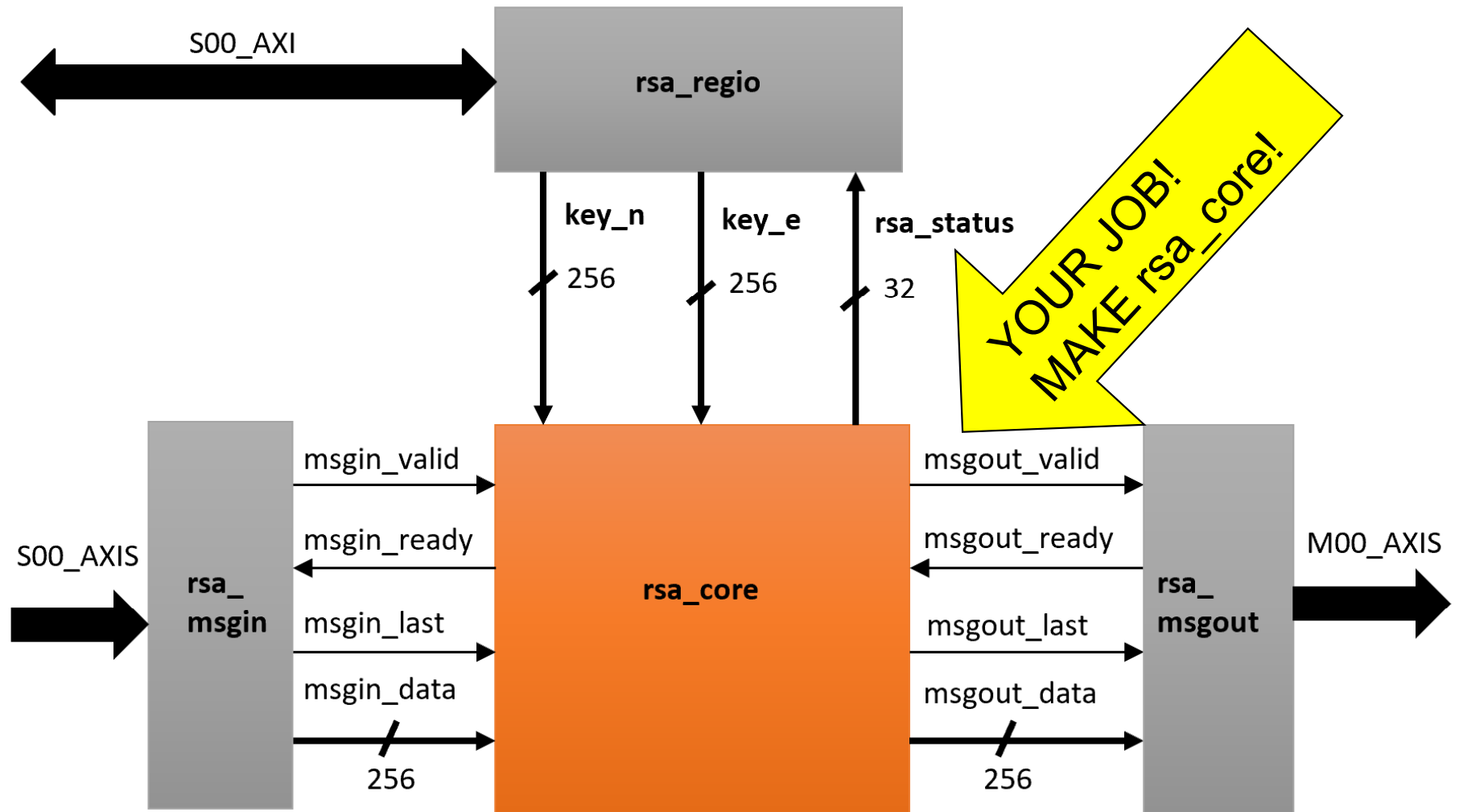
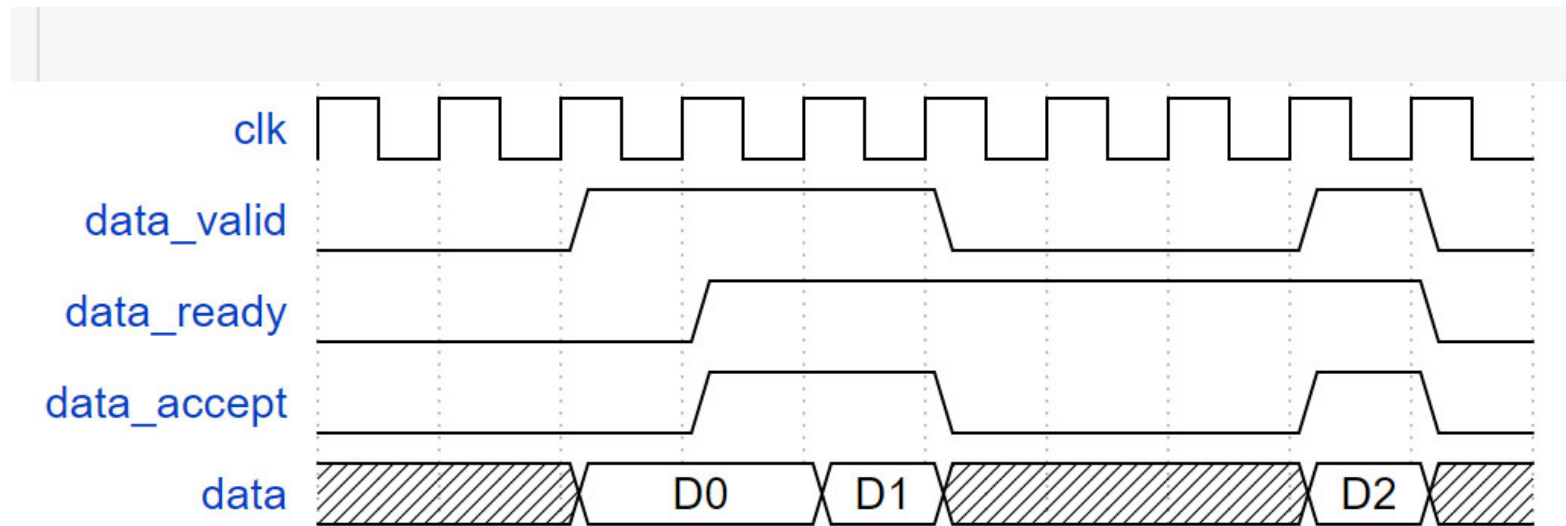


Figure 5. Main blocks within the RSA ACCELERATOR

Interface description

Ready valid handshaking used for transferring information on all interfaces.



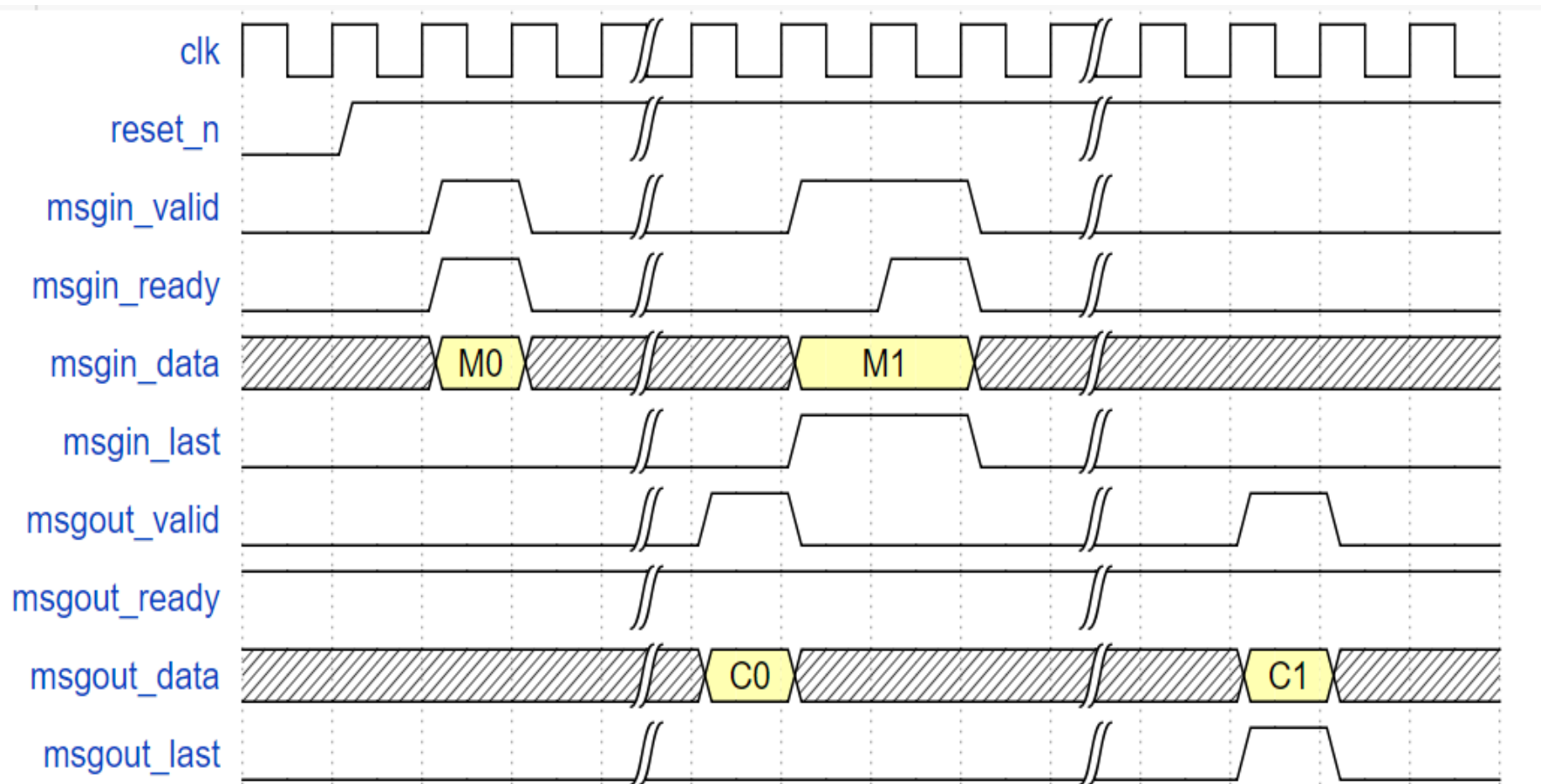
Ready valid handshaking (Chapter 22)

Data is transferred when both valid (from sender) and ready (from receiver) are high at the same time.

This is illustrated with the derived helper signal named `data_accept` (not part of the protocol) in the figure above

The handshaking is exactly the same independent of what information that is transferred. Common information includes *data*, *addresses*, *stream data*, *write responses etc.*

Messages in and out of rsa_core



Memory mapped registers

Address	Register
0-7	Key N ($8 \times 32\text{bit} = 256\text{ bit}$)
8-15	Key E ($8 \times 32\text{bit} = 256\text{ bit}$)
16-23	User defined
24-31	User defined
32	Status register

Proposed use of the memory mapped registers, but this is entirely up to the group to decide.

Some algorithms, e.g., Montgomery, can benefit from precalculating values and put them in user defined registers.

Summary of your task

- **Major Goal - Design and Implement the RSA Circuit:**

The circuit should be able to encrypt and decrypt a message of 256 bits as quickly as possible. Design must fit inside the FPGA.

- **Understand Algorithm and create model**

Literature study of RSA and HW realization of RSA.
Create a high level model of the RSA algorithm.

- **Create microarchitecture for HW design**

Suggest a design that implements RSA according to the specification. Create block diagram for your solution. It must be possible to identify mux-es, registers, adders etc.

- **Write RTL code and test**

Write RTL code for your design based on the microarchitecture.

Summary of your task

- Subsequent works of this project:
 - Simulate and Verify your circuit function
 - Design the system in hardware and perform physical implementation on an FPGA
 - (Circuit shall be tested and verified in the lab)
- Benchmarking
 - After the circuit has been verified and implemented on an FPGA, then it should be ready to put to the test on a performance benchmarks.

Recommended literature

Start by reading:

C. K. Koc, «RSA Hardware Implementation», RSA Laboratories, August 1995 (in particular Section 4 and Section 7.1)

There is much additional literature on Blackboard (much more than you need).



Schedule term project

Milestone	Date	Delivery instructions	Description	Check
Form groups	AUG 26	Sign up on Blackboard	Form term project groups	
Study algorithms and pick one	SEP 10	Nothing to upload	Study algorithms and pick one	
High level model	SEP 17	Upload code on Blackboard	Implement the algorithm in python or another high level language.	
Microarchitecture	OCT 3	Upload diagram on Blackboard	Draw microarchitecture diagram for hardware design in this datasheet.	
Performance estimate	OCT 3	Estimate performance. Upload to Blackboard.	Estimate the time needed to encrypt/decrypt a block, in this datasheet.	
Microarchitecture review/presentation	OCT 3	Give presentation in class.	Staff and fellow students (peers) reviews the solutions proposed by each team and gives feedback.	
RTL Code (Alpha)	OCT 29	Upload RTL code to Blackboard.	Write synthesizable register transfer level code.	
Testbench (Alpha)	NOV 5	Upload Testbench to Blackboard.	Write testbenches for testing the design.	
Working on FPGA (Alpha)	NOV 12	Upload PPA on Blackboard.	Design working on FPGA.	
Hand in this document and all pieces of source code	NOV 21	Upload this document together with all pieces of source code on Blackboard.	Hand in this document	



Design requirements

Requirement ID	Priority	Description
REQ_FUNC_01	MUST	The design must implement a function that can compute modular exponentiation $X = Y^k \bmod n$
REQ_FUNC_02	MUST	The design must be able to encrypt and decrypt message blocks using modular exponentiation: Encryption: $C = M^e \bmod n$, $M < n$, $C < n$, $e < n$ Decryption: $M = C^d \bmod n$, $M < n$, $C < n$, $d < n$
REQ_PPA_01	MUST	Encrypt/decrypt a message of length 256 bits as fast as possible.
REQ_PPA_02	MUST	The design must fit inside the Zynq XC7Z020 FPGA on the Digilent Pynq-Z1 board.
REQ_PPA_03	MUST	There is no requirement for the clock frequency of the programmable logic. The platform supports any clock frequency.
REQ_PPA_04	SHOULD	The hardware accelerator should run testcase 4 faster than 400 ms.
REQ_INT_01	MUST	The RSA design must be integrated as a hardware accelerator inside the Zynq SoC. It must be managed by the CPU and made accessible through the Juniper notebook interface.
REQ_INT_02	SHOULD	The design should implement memory mapped status registers, performance counters and other mechanisms for debugging of features and performance at system level.
REQ_INT_03	MUST	The design must have one AXI-Lite Slave interface to enable access of memory-mapped registers.
REQ_INT_04	MUST	The design must have one AXI stream slave interface for input messages that shall be encrypted(decrypted) and one AXI stream master interface for output messages that have been encrypted(decrypted).
REQ_CONF_01	SHOULD	The design should be optimized for 256 bit block/message/key size.

Documentation requirements

Table 2. RSA Hardware accelerator documentation and code requirements

Requirement ID	Priority	Description
REQ_DEV_01	MUST	The development is broken down into milestones. The group must deliver the milestones on time.
REQ_DOC_01	MUST	All green parts of this document must be filled out.
REQ_DOC_02	MUST	This document must contain information about algorithm used for computing modular multiplication.
REQ_DOC_03	MUST	This document must contain description of the design including microarchitecture diagrams.
REQ_DOC_04	MUST	This document must contain verification plan.
REQ_DOC_05	MUST	This document must contain results from performance measurements.
REQ_CODE_01	MUST	RTL code for the design must be attached the final delivery bundle.
REQ_CODE_02	MUST	Code for the testbench(es) developed by the group must be attached the final delivery bundle.
REQ_CODE_03	MUST	High level model code (Python, Matlab, C++) developed by the group must be attached the final delivery bundle.